

---

# PlaceAuth

A common language for machines and places.

WHITEPAPER

# From Permission to Admission

An Open Interoperability Model for Autonomous Systems  
in Physical Environments

**Braden Russell Glasgow**

Project Lead, PlaceAuth

Revision 0.1 | September 2026

SPP 0.3.0 Experimental Preview

Experimental / Pre-standardization

[placeauth.org](https://placeauth.org)

Certain technologies described by PlaceAuth are patent pending.

PUBLICATION INFORMATION

# PlaceAuth / SPP

|                              |                                                                                                                 |
|------------------------------|-----------------------------------------------------------------------------------------------------------------|
| PROJECT                      | PlaceAuth                                                                                                       |
| AUTHOR                       | Braden Russell Glasgow                                                                                          |
| ROLE                         | Project Lead, PlaceAuth                                                                                         |
| PROTOCOL                     | Spatial Policy Protocol (SPP)                                                                                   |
| CURRENT RELEASE              | SPP 0.3.0 Experimental Preview                                                                                  |
| REPOSITORY                   | <a href="https://github.com/placeauth/spatial-policy-protocol">github.com/placeauth/spatial-policy-protocol</a> |
| WEBSITE                      | <a href="https://placeauth.org">placeauth.org</a>                                                               |
| GENERAL                      | <a href="mailto:hello@placeauth.org">hello@placeauth.org</a>                                                    |
| STANDARDS & INTEROPERABILITY | <a href="mailto:standards@placeauth.org">standards@placeauth.org</a>                                            |
| SECURITY                     | <a href="mailto:security@placeauth.org">security@placeauth.org</a>                                              |
| LICENSE                      | Apache License 2.0                                                                                              |
| STATUS                       | Experimental / Pre-standardization                                                                              |

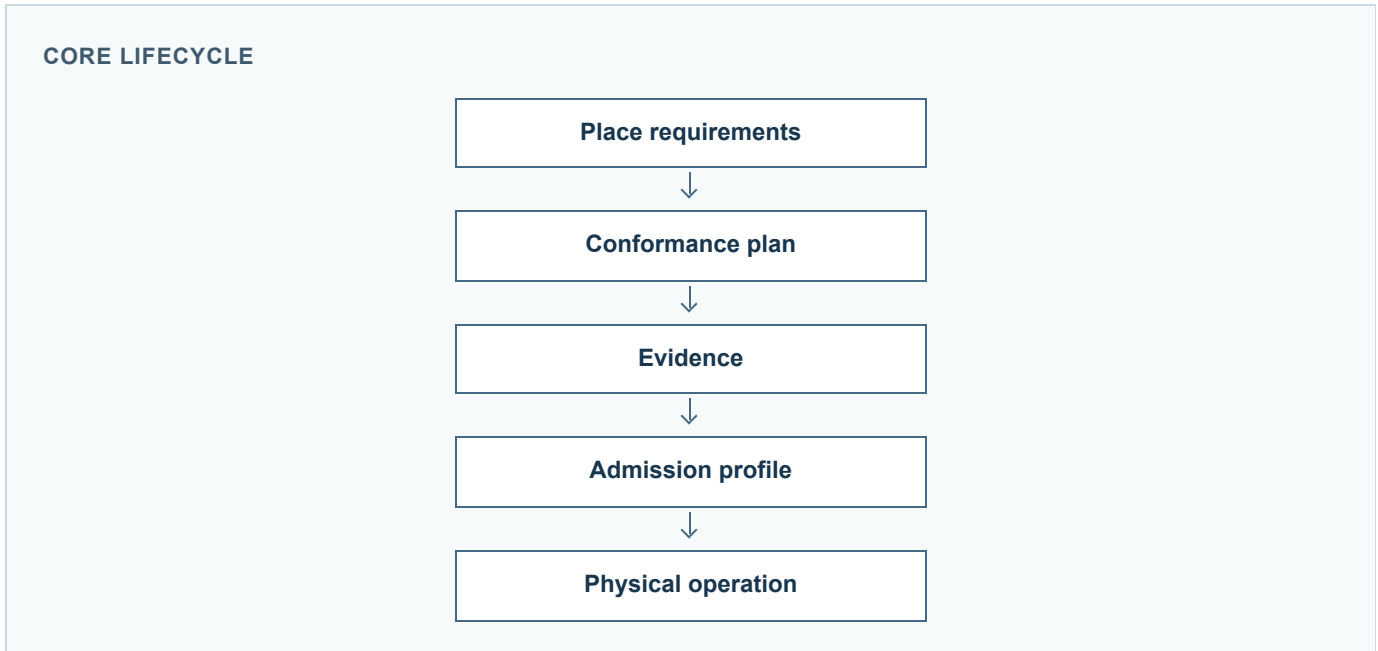
This publication describes an experimental reference implementation and protocol family. It is not a certification, production security system, or adopted standard.

PUBLICATION GUIDE

# Contents

|                                    |                                             |
|------------------------------------|---------------------------------------------|
| 1. Abstract                        | 11. Spatial Context and Transition          |
| 2. Introduction                    | 12. Selective Requalification               |
| 3. The Interoperability Gap        | 13. SPP Architecture                        |
| 4. From Permission to Admission    | 14. Reference Implementation                |
| 5. Place-Defined Requirements      | 15. Security and Trust Model                |
| 6. Embodiment-Neutral Requirements | 16. Relationship to Existing Infrastructure |
| 7. Conformance                     | 17. Open Questions                          |
| 8. Evidence                        | 18. Current Status                          |
| 9. Admission Profiles              | 19. Getting Involved                        |
| 10. Degraded Operation             | 20. Conclusion                              |

# 1 Abstract



Autonomous systems increasingly operate in places that they do not own, design, or control. A delivery robot enters a clinic, a forklift traverses a shared warehouse, a service robot uses an elevator, or a drone approaches a campus. In each case, the place has requirements that are local, contextual, and potentially different from the assumptions encoded in the machine’s own software. A useful interoperability layer must let the place express those requirements without naming a particular vendor or runtime, and must let the machine demonstrate that the relevant conditions are met.

PlaceAuth is the umbrella project for an open interoperability effort in this area. The Spatial Policy Protocol (SPP) is an experimental protocol and specification family between autonomous systems and physical environments. Its core question is:

May Actor X perform Action Y in Space Z under Context C?

SPP separates four concerns that are often coupled in deployments: place-defined requirements, machine conformance, evidence, and the operating profile that results. The place publishes requirements for a hierarchy of spaces. A machine, through an embodiment-appropriate adapter, maps those requirements to tests or other proof mechanisms. Evidence is bound to relevant machine, controller, policy, environment, and plan state, with freshness and assurance information. The admission layer then expresses an operating profile as `ADMITTED`, `DEGRADED`, or `DENIED`.

SPP 0.3.0 Experimental Preview is a reference implementation for technical review and interoperability experimentation. Its reference package version is 0.3.0; the normative protocol specification remains SPP 0.1. Place Package and explain-trace formats remain 0.1. The reference surfaces include machine-readable schemas, YAML examples, deterministic conformance and admission scenarios, a local and OPA/Rego HTTP reference server, versioned requirement semantics, signed place and evidence trust, profile lifecycle assessment, and bounded robot/runtime and place-side integrations. These components demonstrate a coherent model; they do not constitute certification, production security infrastructure, or evidence of industry adoption.

## 2 Introduction

The operating boundary of an autonomous system is no longer necessarily the facility where it was designed or trained. Logistics fleets cross tenants and buildings. Robots are asked to perform tasks in hospitals, hotels, homes, campuses, and warehouses. A machine may be known to its operator but unknown to the authority responsible for the destination space. It may also have a different sensor configuration, controller build, data-retention behavior, or safety envelope from another machine of the same product family.

Physical places already express conditions through signage, operating procedures, access-control systems, safety rules, and building automation. Those representations are useful to people and to local systems, but they are difficult for an external autonomous system to consume in a consistent, vendor-neutral way. Conversely, a robot may be able to demonstrate useful properties, but the property owner has no common vocabulary for requesting them or for deciding how much proof is sufficient.

The missing interaction is not simply another API for commanding a robot. It is a contract between a place and an autonomous actor. The place needs to state what must be true in a particular space and context. The actor needs to show what it can demonstrate at a particular point in time and configuration. The result needs to remain scoped: a demonstration valid in a lobby should not silently become valid in a pharmacy, a patient room, or a human-only aisle.

PlaceAuth develops SPP to make that interaction explicit. PlaceAuth is the umbrella project; SPP is the protocol and specification family. PlaceAuth is not a formal standards body. SPP is an experimental open interoperability protocol, intended to be reviewed, implemented, and challenged by others before any standardization path is considered.

## 3 The Interoperability Gap

Several distinct questions are often collapsed into a single permit or deny result:

- **Place requirements:** What speed, sensing, data, manipulation, infrastructure, or human-interaction conditions apply in this space?

- **Robot capabilities:** What can this machine, controller, sensor configuration, and runtime actually do?
- **Authorization:** Is this actor or purpose allowed to request the action?
- **Conformance:** Which test, observation, or attestation corresponds to each requirement?
- **Evidence:** What was demonstrated, when, against which policy and environment, and with what assurance?
- **Operating conditions:** What restrictions, expiry, or changed boundaries apply while the machine acts?

An identity assertion can answer who is requesting an action. It does not establish that the current machine state satisfies a local operating condition. A policy rule can state that video recording is denied. It does not by itself prove that a robot has disabled capture or cannot retain the resulting data. A conformance result can show that a test passed. It does not remain valid indefinitely if the policy, environment, controller, or relevant configuration changes.

Without a common model, each integration invents private terminology and point-to-point assumptions. A building system may know a door and an elevator, while a fleet system knows a robot identifier and a task. A safety system may know separation distance, while a data-governance system knows retention. The same requirement may be described differently by each vendor, and a deployment may have no durable way to inspect why an action was admitted.

SPP does not attempt to replace those systems. It provides a place-facing exchange that can carry requirements, decisions, proof references, and operating conditions across their boundaries. Its value is in making the relationship between the questions visible and testable.

## 4 From Permission to Admission

The place defines the requirements.  
The machine demonstrates conformance.  
SPP establishes the operating profile.

Traditional authorization asks:

May this machine perform this action?

That question remains necessary. SPP Core evaluates an actor, action, space, and context against place policy and returns `permit`, `deny`, or `conditional`. A conditional result is not a weak permit: the enforcement point must block the action until the stated requirements are satisfied and the request is evaluated again.

Evidence-based admission asks a broader operational question:

## What guarantees has this machine demonstrated, and under what conditions may it operate here?

The distinction is important because a place may need to reason about properties that are not represented by an identity or a simple role. It may require a maximum speed, a minimum distance from people, disabled facial recognition, zero video retention, or a constrained interaction with a door or elevator. The answer should state not only whether an action is allowed, but also which guarantees support that answer, which restrictions apply, and when the answer expires.

SPP treats admission as a spatially scoped operating profile. The profile is derived from the applicable requirements and the evidence available for this actor and context. It does not assert that the machine is universally safe or trustworthy. It records a bounded result for a bounded place, action set, and validity window.

## 5 Place-Defined Requirements

The authority for a place defines requirements in terms of the place and the operation, rather than the internal architecture of a visiting machine. Illustrative requirements include:

- movement speed, traversal, docking, dwelling, or exit;
- sensing such as lidar, video capture, audio recording, or thermal observation;
- data collection, transmission, retention, export, or erasure;
- manipulation such as grasping, dispensing, touching, or package placement;
- infrastructure interaction such as doors, elevators, charging, or building systems; and
- human interaction such as approach, identification, notification, assistance, or contact.

SPP 0.1 defines six initial action families: `movement`, `sensing`, `data`, `manipulation`, `infrastructure`, and `human_interaction`. Action names remain deployment-defined. For example, `movement.enter`, `sensing.video_capture`, and `infrastructure.elevator_use` are illustrative names, not a universal registry.

Requirements can include selectors for actor identity or type, context, required authorizations, obligations, reasons, and decision lifetime. An obligation constrains an otherwise permitted activity. If an enforcement point cannot understand or apply an obligation such as no-retention, it must deny rather than silently ignore it.

Spaces form an explicit hierarchy. Each declared space has an optional parent, and the parent graph must be acyclic. A child space inherits applicable parent rules, while a matching child rule can override an inherited rule for that action. Slash-separated identifiers may help people read a policy, but the explicit `parent` property is authoritative. Unknown spaces and unmatched actions fail closed to denial.

The following is illustrative policy notation, not a complete deployment file:

```
space: hospital.example/floor/3
parent: hospital.example
policy:
  movement:
    enter: permit
  sensing:
    video_capture: deny
  data:
    video_retention: deny
  infrastructure:
    elevator_use: permit
  manipulation:
    pharmacy: conditional
```

The policy describes what applies in the place. It does not name a particular robot vendor, operating system, middleware, or controller.

For independently implementable conformance, the reference vocabulary uses lowercase dotted requirement identifiers, explicit requirement versions, canonical units, and documented comparison semantics. An extension uses the form `x-<organization>.<requirement>`. Unknown or incompatible requirement semantics fail closed or become `UNRESOLVED`; they are not compared by guesswork. This is intentionally a small interoperability convention, not a universal robotics ontology, remote registry, or unit-conversion system.

## 6 Embodiment-Neutral Requirements

A place should not need to know whether the visiting system is a wheeled base, humanoid, forklift, drone, or another embodiment in order to state the condition it cares about. “Keep at least 1.2 metres from people” is a place requirement. The proof mechanism may differ between a mobile robot using a navigation stack, a humanoid using a locomotion controller, a forklift using a safety PLC, and a drone using a flight controller.

This separation has two advantages. First, it allows a place to publish requirements before choosing a vendor or fleet. Second, it lets an adapter translate the same abstract requirement into an embodiment-appropriate test without exposing proprietary implementation details. The protocol can exchange a demonstrated guarantee and its assurance rather than source code, model weights, or controller internals.

Embodiment neutrality does not mean that every machine can satisfy every requirement. An adapter must be able to state whether it can test a requirement and what evidence it can produce. A place may require stronger assurance than a declaration. A deployment may also impose actor or authorization selectors in addition to abstract operational conditions. SPP leaves those trust decisions to the spatial authority and its deployment mechanisms.

## 7 Conformance

Conformance connects a requirement to a proof mechanism. The experimental model represents this connection as a `ConformancePlan` containing selected tests, policy and environment digests, a challenge, unresolved guarantees, and required assurance. Each `ConformanceTest` is an adapter-level test mapped from one abstract requirement.

The mapping is deterministic in the reference implementation. Given the same requirement set and adapter, the plan selects the same requirement-to-test mapping and produces an inspectable set of unresolved guarantees. Determinism makes the result reproducible; it does not make the test universally sufficient or equivalent to independent certification.

Illustrative mappings include:

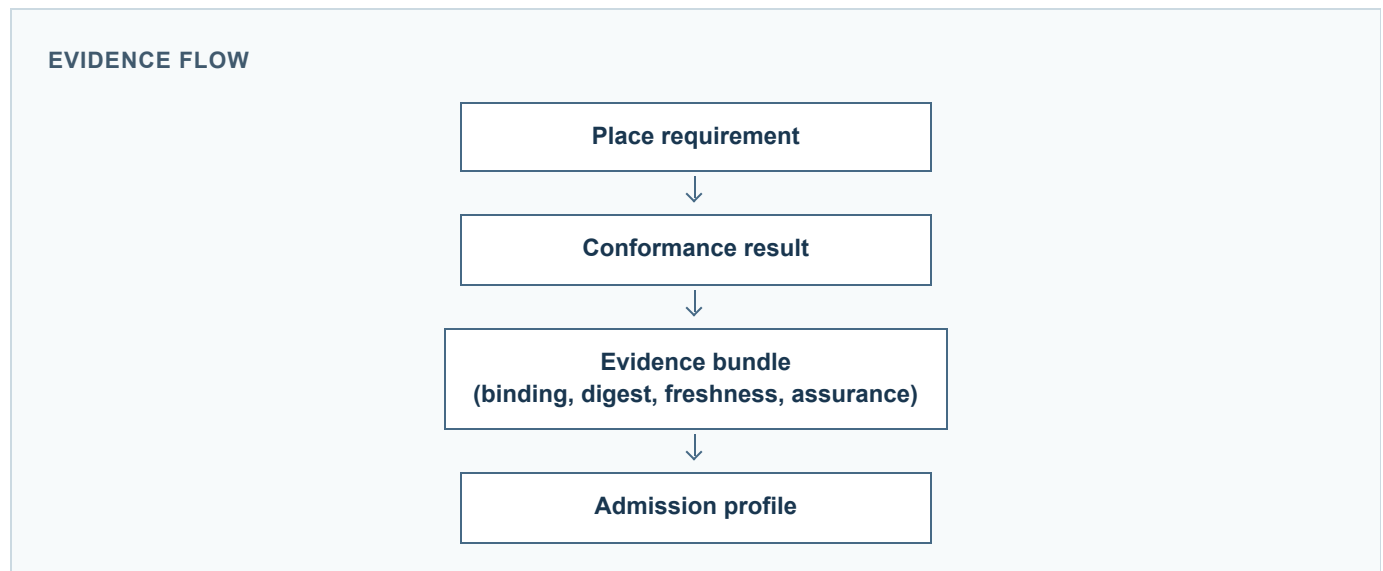
- a mobile robot test that exercises a configured speed limit;
- a humanoid controller test that measures separation during an approach;
- a forklift safety-PLC test for a human-only aisle; and
- a drone flight-controller test for a no-recording or geofenced operation.

These examples are illustrative. The reference implementation includes an embodiment-specific conformance provider registry with mobile and humanoid providers. Its demo implements a simple mobile-base path for speed, human separation, facial-recognition disablement, and zero video retention. The other embodiment examples are not claims of a complete deployment or certification.

The hardened provider contract makes that boundary inspectable. A provider declares its ID and version, exact supported requirement ID/version pairs, supported embodiments, assurance levels, evidence type, and deterministic priority. Selection proceeds by exact requirement/version, embodiment, minimum assurance, priority, and provider-ID tie-break. Provider output is validated against the selected descriptor and vocabulary before it can become evidence; invalid output fails closed. A conformance provider performs a test. It is not automatically an evidence issuer or a policy authority.

Conformance is not the same as authorization. A test can show that a property was demonstrated; the policy still determines whether the action is allowed in the requested space and context. Similarly, a passing test does not grant an indefinite right to operate. Evidence remains bound to policy, environment, actor state, and validity conditions.

## 8 Evidence



An `EvidenceBundle` records the results selected by a conformance plan. An `EvidenceBinding` associates those results with the relevant actor or robot build, controller configuration, policy version and digest, environment or site-model digest, plan state, challenge, and validity window. The purpose is to make applicability inspectable and to reject evidence that is stale, modified, replayed, or bound to the wrong state.

The reference layer supports signed Ed25519 evidence. A signed bundle carries an issuer identifier, signature algorithm, and signature bytes over canonical evidence content. A local `TrustedIssuerRegistry` performs issuer lookup, public-key verification, and authorization for the declared evidence type and scope. The verified admission path fails closed for unknown, disabled, unauthorized, or invalid issuers and signatures. Legacy unsigned evidence remains an explicit compatibility path only; the verified path does not silently treat unsigned evidence as trusted.

The reference implementation demonstrates behavioral evidence at assurance level E2. The schema reserves a range from E0 declaration through E4 trusted external observation, but the preview does not implement all levels. The signed-evidence layer is a local reference trust model, not custom cryptography, a PKI, remote trust discovery, certificate lifecycle management, or a production trust-anchor system.

Evidence freshness matters because a result can become invalid without the machine changing its identity. A controller update, sensor reconfiguration, changed policy, changed site model, or expired validity window can invalidate a previously applicable guarantee. A challenge or nonce helps prevent simple replay within the deployment's acceptance window. The reference registry is intentionally lightweight and in-memory; it is reference/demo infrastructure, not a distributed replay service. At the admission boundary, the implementation revalidates the exact current conformance plan, policy, and environment bindings and rejects time-of-check/time-of-use changes that occur after initial evidence evaluation.

The evidence model is deliberately proportional. It exchanges a guarantee, result, binding, digest, and assurance level. It does not require a robot to disclose proprietary internals. At the same time, a digest is not a magical proof of physical truth. A compromised adapter, dishonest self-report, or compromised runtime can still produce misleading evidence. The enforcement point and deployment trust architecture remain essential.

Place requirements are verified separately. A signed Place Package 0.1 is a portable JSON exchange containing place identity, policy identity and version, scope, authority identity, versioned requirements, canonical units, and a deterministic digest and Ed25519 signature. A local policy-authority registry verifies the authority's place/scope authorization. Tampering with a requirement ID, version, value, or unit changes the signed package content and invalidates verification. Known legacy built-ins may resolve deterministically to documented version 1.0 definitions; unknown extensions require explicit local registration. The format does not provide network discovery, a global registry, or proof that an authority legally or physically controls a place.

## 9 Admission Profiles

### ADMISSION PROFILE OUTCOMES

| Profile  | Meaning                                                                  |
|----------|--------------------------------------------------------------------------|
| ADMITTED | Required guarantees have been demonstrated within the evaluated scope.   |
| DEGRADED | Operation may continue only with explicit, enforceable restrictions.     |
| DENIED   | The requested operation must not proceed under the evaluated conditions. |

The Admission layer evaluates the applicable requirements together with the available evidence and produces a spatially scoped operating profile.

**ADMITTED** means the required guarantees for the evaluated profile have been demonstrated at the required assurance and remain valid for the stated actor, space, policy, environment, and time window. The profile should identify those guarantees and any obligations that the enforcement point must apply.

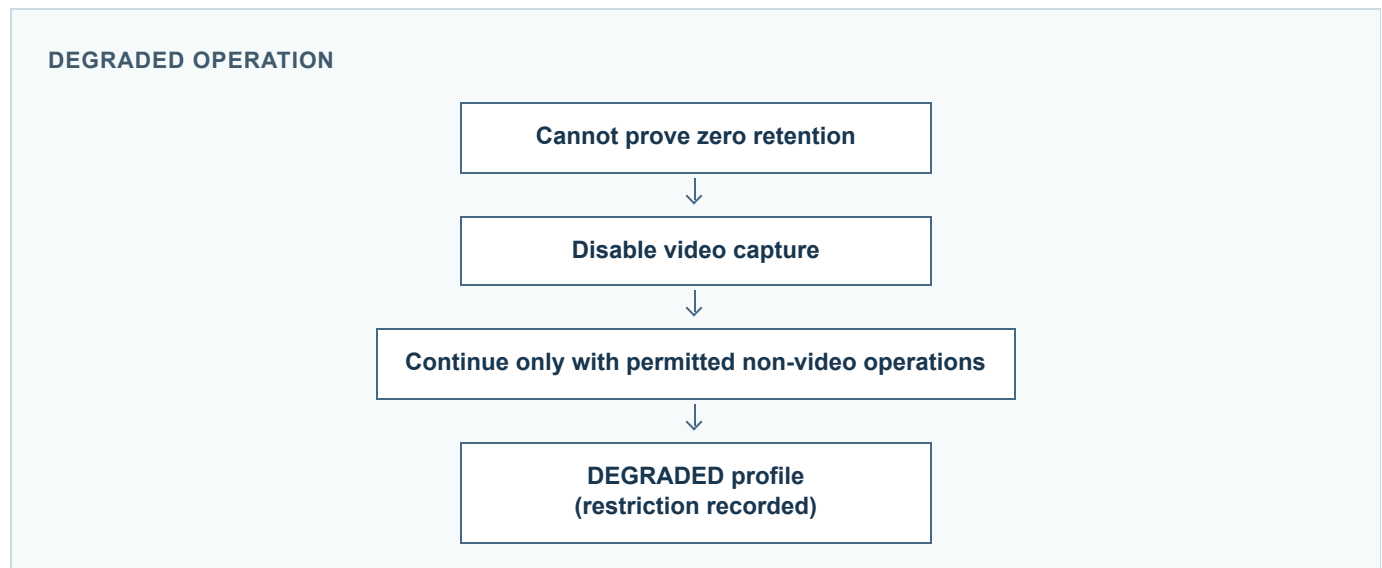
**DEGRADED** means the machine may continue only under explicit restrictions because a non-essential requirement was not demonstrated or is unavailable. A degraded profile is not a full permit. It must name the restriction and the behavior that remains allowed.

**DENIED** means the machine must not perform the requested operation under the evaluated conditions. Essential safety failures, malformed or stale evidence, binding mismatches, unsupported obligations, unknown spaces, and other fail-closed conditions belong here unless a deployment has a separately reviewed mechanism that changes the request and evaluates it again.

An admission profile is not a certification and does not generalize beyond its scope. A profile can expire, be superseded by a policy version, or require reevaluation when the actor, action, space, context, authorization state, or environment changes.

Lifecycle assessment is read-only: it does not silently mutate a profile. It returns `VALID`, `REVALIDATE`, `REQUALIFY`, or `INVALID`. Evidence expiry or controller changes require revalidation; changed policy, environment, destination, new or stricter requirements require requalification; actor/build changes, disabled trust anchors, explicit local revocation, or failed current trust checks invalidate the profile. An unresolved lifecycle state must not be treated as valid. This reference model does not claim distributed revocation or continuous attestation.

## 10 Degraded Operation



Degraded operation gives a deployment an explicit response to a non-essential gap without converting that gap into an unqualified permit. Consider a place requirement that video retention must equal zero. If the machine cannot demonstrate zero retention, the safe response may be:

```
cannot prove zero retention
-> disable video capture
-> continue only with the permitted non-video operations
-> issue a DEGRADED profile with the restriction recorded
```

The restriction must be enforceable. If the enforcement point cannot actually disable capture or guarantee no retention, it must deny the action. A degraded profile cannot be used to bypass a human-separation requirement or another essential safety condition. The reference implementation fails closed on essential safety failures and records explicit degraded restrictions for non-essential failures.

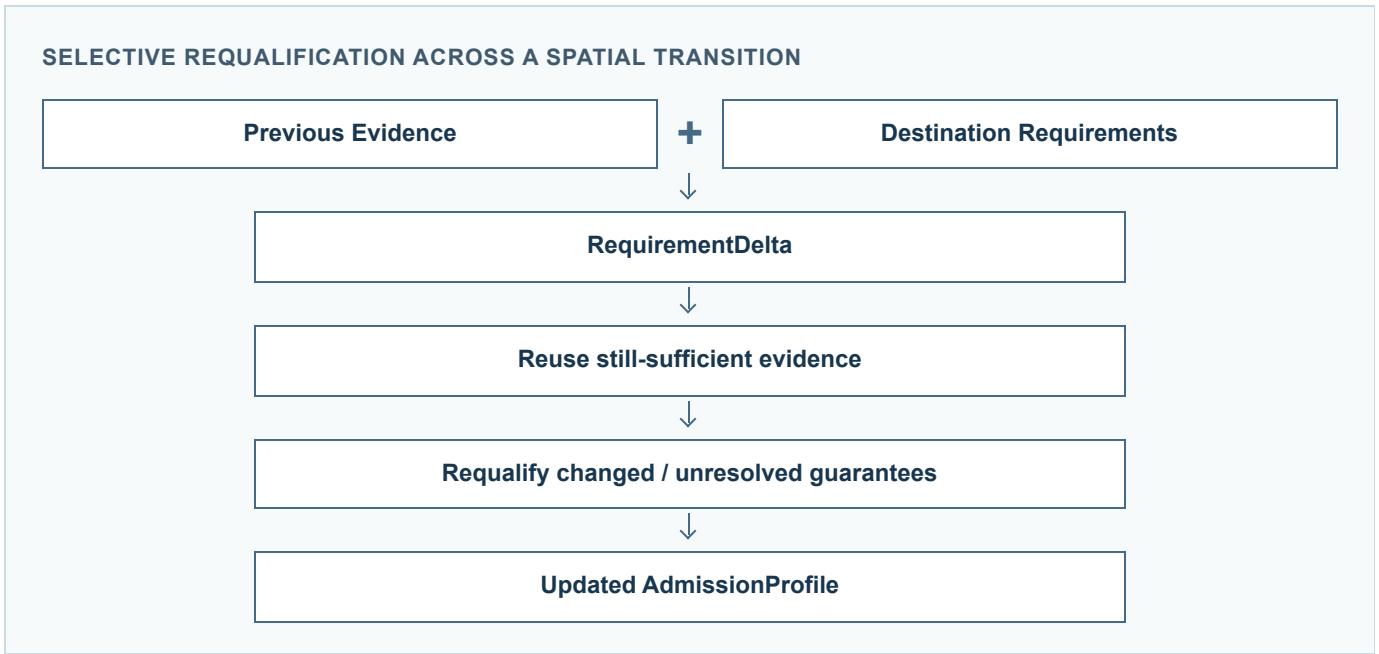
## 11 Spatial Context and Transition

Requirements belong to spaces, and different spaces can impose different conditions on the same actor. A lobby may allow a robot to enter at a configured speed. A patient wing may add human-separation, facial-recognition, and video-retention requirements. A pharmacy may additionally require staff authorization. A human-only warehouse aisle may deny traversal even when the same machine is permitted elsewhere.

The requested space is resolved through its explicit parent chain. The nearest matching rule takes precedence for the requested action, while unrelated actions continue walking toward the parent. Context can carry purpose, time, emergency state, named authorizations, and deployment attributes. Authorization strings supplied by an actor are not trusted until the deployment verifies the underlying credential or grant.

This spatial model keeps the operating profile local. A lobby result cannot be reused in a pharmacy merely because the actor identifier is unchanged. A decision should be reevaluated when the space changes, when its policy version changes, when the authorization state changes, or when the decision lifetime expires.

## 12 Selective Requalification



Selective requalification is the central transition pattern in the experimental admission model. It avoids treating every spatial transition as either a complete retest or an unconditional carry-over. Instead, the prior guarantees are compared with the destination requirements.

Consider a robot moving from a lobby to a patient wing:

```
Previous evidence (Lobby)
  movement.max_speed <= 0.8

Destination requirements (Patient Wing)
  movement.max_speed <= 0.8
  human_separation >= 1.2
  sensing.facial_recognition = prohibited
  data.video_retention = 0

Comparison
  movement.max_speed      -> REUSED
  human_separation        -> NEW
  facial_recognition       -> NEW
  video_retention         -> NEW

Reduced conformance plan
  run only the three unresolved tests

Updated operating profile
  ADMITTED if the new guarantees pass
```

The transition can also classify requirements as `STRICTER`, `RELAXED`, `NO_LONGER_APPLICABLE`, or `UNRESOLVED`. Requirements are compared only when their ID/version semantics are compatible. An incompatible version is `UNRESOLVED`, rather than a value that can be compared as though equivalent. A stricter destination condition must not be silently treated as the old guarantee. A relaxed condition may be satisfied by an existing stronger result, subject to the implementation’s comparison rules. A requirement that no longer applies should not remain as an unexplained restriction in the new profile.

Selective requalification does not weaken the place’s authority. It makes the reason for reuse explicit and preserves the boundary between proven and unproven conditions. The destination supplies the requirements; the actor supplies evidence; the admission layer derives the `RequirementDelta`, reduced conformance plan, and updated profile. The robot application can remain unchanged while the place changes the conditions under which it may operate.

## 13 SPP Architecture

### SPP PROTOCOL SURFACES

| Surface         | Role                                                          |
|-----------------|---------------------------------------------------------------|
| SPP Core        | Place-facing policy and decision contract.                    |
| SPP Conformance | Requirement-to-proof mapping and deterministic plan.          |
| SPP Admission   | Evidence verification and spatially scoped operating profile. |

The public architecture is intentionally small:

```
PlaceAuth
└─ Spatial Policy Protocol (SPP)
    ├── SPP Core
    ├── SPP Conformance      (experimental)
    └─ SPP Admission        (experimental)
```

**SPP Core** defines the place-facing policy and decision contract: actors, actions, spaces, context, hierarchical inheritance, rules, obligations, and `permit / deny / conditional` outcomes. It does not define identity proofing, discovery, geometry, robot control, transport authentication, policy signatures, revocation distribution, or audit storage.

**SPP Conformance** turns abstract place requirements into a deterministic plan of adapter-level tests or other proof mechanisms. It records the requirement-to-proof mapping, unresolved guarantees, policy and environment references, and the assurance level requested by the place.

**SPP Admission** verifies evidence and derives the operating profile. It checks binding, freshness, integrity, signed-evidence issuer trust when the verified path is selected, policy and environment state, challenge use, and the result required for each guarantee. It also models `RequirementDelta` during spatial transitions, supports selective requalification, and revalidates the current admission boundary before a profile is used.

The layers are protocol surfaces, not necessarily separate network services. A deployment may combine them in one process or distribute them across policy, fleet, robot, and building components. The reference repository keeps the boundaries visible so that independent implementations can replace individual pieces.

An independent implementation can follow the published exchange path without depending on the Python reference code:

```
Place Package -> Requirement vocabulary -> Conformance provider -> Evidence
               -> Admission -> Profile lifecycle -> Decision trace
```

The practical implementation guide documents this flow and its fail-closed boundaries. The schema-defined, deterministic decision trace uses `trace_version: "0.1"` and can contain place and subject summaries, requirements, provider selection, evidence assessment, `RequirementDelta`, selected tests, reused guarantees, admission, lifecycle, restrictions, and reasons. It explains a decision; it is not authorization or an independent proof of physical behavior.

## 14 Reference Implementation

The current reference implementation demonstrates:

- JSON Schemas for policy, request, and decision data;
- YAML examples for home, hospital, warehouse, and hotel spaces;
- hierarchical policy inheritance and deny-by-default evaluation;
- deterministic conformance planning and requirement-to-proof mapping;
- evidence generation and binding to relevant state;
- evidence digest, freshness, policy/environment binding, reference replay checks, and signed Ed25519 evidence with local trusted-issuer verification;
- signed Ed25519 place requirements and Place Package 0.1 with local trusted-policy-authority verification;
- dotted, versioned requirement vocabulary with canonical units and fail-closed compatibility handling;
- external conformance-provider descriptors and deterministic validated provider-result conversion;
- `ADMITTED`, `DEGRADED`, and `DENIED` admission profiles;
- essential-safety denial and explicit degraded restrictions;
- spatial transition deltas, version-aware `RequirementDelta`, selective requalification, and profile lifecycle assessment;
- local policy evaluation and OPA/Rego integration;
- a FastAPI reference server exposing health and decision endpoints;
- an experimental ROS 2/Nav2 speed-limit adapter with `SpeedLimit` runtime, `ControllerServer`-to-plugin-boundary, and stock Regulated Pure Pursuit motion validation;
- live Open-RMF task-eligibility validation at the `FleetUpdateHandle.consider_delivery_requests` boundary;
- a bounded facility-side access-control decision boundary; and
- deterministic clinic and admission demonstrations, including schema-defined `spp-explain` decision traces.

The dependency-free reference test suite reports **255 passed, 5 skipped** in the documented environment. One unrelated Windows pytest-cache warning may appear in that environment. The four admission scenarios are:

```

A  full conformance          -> ADMITTED
B  video-retention failure    -> DEGRADED
C  essential safety failure   -> DENIED
D  transition and requalification-> updated profile

```

The count describes the state of the reference test suite. It is not a measure of adoption, certification, interoperability across independent vendors, or production readiness. The ROS 2 component is an experimental speed-limit adapter. In a ROS 2 Humble / Nav2 runtime fixture, an evidence-backed operating profile changed commanded navigation speed from 1.0 m/s to 0.5 m/s under the same active `FollowPath` goal, using `nav2_msgs/msg/SpeedLimit`, `ControllerServer`, and stock Regulated Pure Pursuit. This is not a physical-robot safety claim, a guaranteed-stopping claim, physical-world safety validation, functional-safety certification, or proof of production enforcement.

SPP task eligibility has also been validated through a live Open-RMF Humble

`FleetUpdateHandle.consider_delivery_requests` runtime path using RMF fleet adapter Python 2.1.8, a real Adapter, a real `FleetUpdateHandle`, one stationary registered test robot, and real ROS delivery bids. The delivery-consideration callbacks were invoked four times: `ADMITTED` produced a bid proposal; `DENIED` produced no proposal with `admission_denied`; `DEGRADED` was not runtime-tested. This does not establish physical robot dispatch execution, traffic negotiation, fleet-wide production integration, or physical motion. The OPA/Rego path is a reference adapter. Deployments must supply their own identity, transport, enforcement, safety, availability, and trust mechanisms.

The facility-side reference boundary demonstrates the other side of the exchange: `AdmissionProfile` -> facility access-control decision. An `ADMITTED` profile for the matching place may grant access. A `DEGRADED` profile grants only when every restriction is explicitly accepted. `DENIED`, wrong-place, stale, revoked, or invalid-lifecycle profiles deny. This is a generic decision boundary only; it does not validate physical door hardware, BACnet, MQTT, a vendor access-control system, or physical-security certification.

For operational review, the reference `spp-explain` command produces a deterministic trace of the evaluated request, resolved policy path, applicable requirements, evidence and admission checks, and resulting decision or operating profile. The trace is an explainability aid for the reference behavior; it is not an audit system or an independent proof of physical behavior.

## 15 Security and Trust Model

SPP does not itself force a malicious autonomous system to comply.

**SPP does not itself force a malicious autonomous system to comply.** A syntactically valid decision or admission profile is not sufficient evidence that a physical action is safe.

Trust depends on the deployment's:

- evidence assurance and provenance;
- actor identity and credential verification;
- runtime and controller integrity;
- decision and policy enforcement points;
- robot and building actuator isolation;
- site infrastructure and trusted localization;
- hardware guarantees;
- attestation, observation, or independent monitoring mechanisms; and
- operational controls such as emergency stops and collision avoidance.

The security guidance requires authenticated authorities and actors, integrity-protected transport, protected policy administration, short-lived decisions, trusted time, fail-closed behavior on errors, and minimum necessary context. The reference layer keeps three roles distinct: a **policy authority** defines trusted place requirements; a **conformance provider** performs a test; and an **evidence issuer** attests the resulting evidence. Signed place requirements use Ed25519 with local authority place/scope authorization. Signed evidence uses Ed25519 with local issuer evidence-type/scope authorization. The model verifies issuer or authority identity and canonical integrity at those boundaries; it does not make one role trusted for another. Logs may still reveal sensitive information about facilities, people, robots, and denied activity.

The threat model includes policy tampering and rollback, actor spoofing, context and space forgery, action confusion, decision replay, obligation stripping, parser differentials, denial of service, policy probing, and enforcement bypass. Evidence-based admission adds forged or modified test results, stale or replayed bundles, wrong-build or wrong-configuration evidence, compromised adapters, unavailable attestation, and unsafe degraded profiles treated as full admission.

SPP 0.1 does not standardize a PKI, remote trust discovery, certificate chains or lifecycle, distributed revocation or replay protection, geometry, or certification. The reference trust layers do not provide production key management, HSM protection, certificate lifecycle, distributed revocation infrastructure, global federation, or proof of legal authority or physical trust in the evidence-generation process. Production deployments must provide appropriate mechanisms. Safety systems remain independently authoritative and can override an SPP permit. These limitations are part of the model, not implementation details to be hidden by a positive profile.

## 16 Relationship to Existing Infrastructure

SPP is intended to complement existing systems rather than replace them. A deployment may use ROS 2 for robot communication, Open-RMF for fleet and building-resource coordination, building-automation systems for doors and elevators, identity systems for actor credentials, policy engines for evaluation, localization systems for space context, and attestation technologies for stronger evidence.

SPP contributes the place-facing vocabulary and exchange: requirements, applicable spaces, decision semantics, proof references, evidence bindings, and operating profiles. A ROS 2 node or fleet adapter can act as an enforcement point. An OPA/Rego policy engine can evaluate a policy representation. A building system can remain authoritative for an actuator. A localization service can provide context that the decision point treats as trusted only to the degree the deployment supports.

This composability is deliberate. SPP does not assume one middleware, transport, robot form, policy engine, or building vendor. Nor does it claim that an integration exists merely because a system is named as compatible. The current ROS 2 integration is an experimental speed-limit adapter with bounded Humble runtime validation through `nav2_msgs/msg/SpeedLimit`, `ControllerServer`, and stock Regulated Pure Pursuit; it observed a 1.0 m/s to 0.5 m/s command-speed change under the same `FollowPath` goal, not stock-controller safety or physical enforcement. The Open-RMF integration has bounded live Humble task-eligibility validation at `FleetUpdateHandle.consider_delivery_requests` using a stationary test robot and real delivery bids, not physical dispatch execution or motion. The facility-side adapter is a generic AdmissionProfile-to-access-decision boundary, not a building-system integration. The preview contains no formal affiliation or endorsement by the projects mentioned here.

## 17 Open Questions

The experimental model leaves important questions for future technical work and review:

1. **Identity and trust anchors:** The reference layer verifies named local issuers; how should places, actors, fleets, and evidence issuers discover and authenticate one another across organizations?
2. **Evidence assurance:** Which assurance levels are meaningful for different requirements, and how should independent observation or hardware attestation be represented?
3. **Physical enforcement:** How can an admission profile be coupled reliably to the actuator, runtime, or building controller that must enforce it?
4. **Discovery:** How does a machine find the authoritative policy for the space it is entering without exposing unnecessary facility information?
5. **Multi-authority policy:** How should conflicting requirements from landlords, tenants, hospitals, fleet operators, or emergency authorities be combined?
6. **Interoperability testing:** What conformance suites and shared fixtures are needed for independent implementations to compare behavior?
7. **Privacy:** What is the minimum evidence and context needed without disclosing sensitive identity, health, location, or operational information?
8. **Standardization path:** Which parts of the model are stable enough for a future standards process, and which should remain deployment-specific extension points?

These questions are intentionally open. The preview provides a concrete vocabulary and runnable reference behavior so that discussion can be grounded in implementations rather than in an assumed final architecture.

## 18 Current Status

SPP 0.3.0 is an **Experimental Preview** developed under PlaceAuth. Its reference package version is 0.3.0, while the normative protocol specification remains SPP 0.1. Place Package and explain-trace formats remain 0.1. It is available in a public repository under the Apache-2.0 license and is positioned for pre-standardization technical review. The repository includes specifications, schemas, examples, reference services, independently implementable interoperability guidance, experimental runtime and place-side boundaries, demos, tests, security guidance, and release notes.

The preview is usable as a reference and experimentation surface, but it is not production-ready security infrastructure. In particular, it does not provide a production PKI, remote trust discovery, production key management, distributed replay protection, hardware attestation, certification, production-grade ROS 2/Nav2 or Open-RMF deployment integration, physical enforcement guarantees, discovery, or broad vendor interoperability testing.

Technical feedback is welcome when it is specific and reproducible: a schema ambiguity, an evaluation discrepancy, a security concern, an interoperability proposal, or a failing test is more useful than an assumption that the preview already represents a finished standard.

## 19 Getting Involved

The public repository is:

<<https://github.com/placeauth/spatial-policy-protocol>>

Contributors and reviewers can help by:

- reviewing the SPP Core, Conformance, and Admission terminology;
- reproducing the deterministic demos and test suite;
- testing schemas and policy examples with independent implementations;
- reporting implementation issues with a minimal reproduction;
- proposing interoperability extensions with examples and focused tests; and
- reporting security concerns through the repository's documented security channel.

Feedback should distinguish protocol behavior from reference-implementation behavior. A proposed extension should explain its spatial scope, interaction with hierarchy and inheritance, assurance expectations, and failure mode. Security reports should avoid exposing sensitive facility or personal information in public issues.

## 20 Conclusion

Autonomous systems will increasingly cross boundaries defined by organizations other than their operators. A place needs a way to state the conditions that matter there without depending on a visiting machine's vendor or internal architecture. A machine needs a way to demonstrate that those conditions are met, with evidence whose scope, freshness, and limitations can be inspected. The result needs to remain local to the relevant space and context, enforceable by the systems that control physical action, and explicit about uncertainty.

The core idea is simple:

Physical environments should be able to state what must be true. Autonomous systems should be able to demonstrate that those conditions are met. The result should be a machine-readable operating profile that both sides understand.

SPP 0.3.0 is an experimental implementation release that makes that exchange more independently implementable while the normative protocol specification remains SPP 0.1. Its next value comes from review, independent implementations, careful security analysis, and evidence about where the model is useful or incomplete.

Certain technologies described by PlaceAuth are patent pending.